Project 2 Discussion Questions:

1. Compare distance-vector and link-state routing in terms of stored information, control traffic, convergence, and implementation complexity.

- Stored information: In DV, each node only stores its own routing table (dest, next hop, cost). It never sees the whole network. In LS, every node stores the full map of the network (every node's neighbor list).
- Control traffic: In DV, each node only talks to its neighbors and sends its whole table every round. In LS, each node floods its neighbor list to the whole network, but only when something changes (plus occasional refreshes).
- Convergence: DV is slower because news only moves one hop per round. In our capacity test (12-node line) it took about a minute for node 1 to learn about node 12. LS converges faster because once the flooded updates arrive, each node runs Dijkstra itself. DV can also have count-to-infinity problems, which LS doesn't have.
- Complexity: DV was simpler to code. It's one update rule (Bellman-Ford) and doesn't need sequence numbers or a full map. LS needs flooding, an LSA cache with sequence numbers, and Dijkstra. But DV needs extra fixes like poison reverse, INFINITY = 16, and timeouts to deal with loops and stale routes.

2. Give an example explaining why an update from the current next hop must be processed even when it makes the route worse. What goes wrong if your node accepts only smaller costs?

- Example from our alternative path test: node 1 reached node 4 through node 2 with cost 2. When we cut the 2-4 link, node 2 told node 1 "my cost to 4 is 16." Node 1 accepted this because node 2 was its current next hop, withdrew the route, and then switched to the path through node 3 (cost 3).
- If node 1 only accepted smaller costs, it would ignore the 16 because 16 > 2. It would keep sending packets for node 4 to node 2, and node 2 can't reach 4 anymore, so they'd all get dropped. The route would stay wrong until it timed out.
- It's also wrong for normal cost increases. If our next hop's path gets longer, our path through it got longer too, since we use that same path. If we ignore that, our table shows a cost that isn't real, and we'd advertise that wrong cost to our other neighbors too.

3. Use a small example to distinguish simple split horizon, poison reverse, and route poisoning after a failure. Why can poison reverse still permit loops involving three or more nodes?

Example: line 1-2-3-4. Node 2 reaches node 4 through node 3.

- Simple split horizon: node 2 just leaves node 4 out of the advertisement it sends to node 3.
- Poison reverse (what we did): node 2 includes node 4 in its advertisement to node 3, but says the cost is 16. Our poison test log shows this: node 2 sent "dest 4 cost 16" to node 3 and "dest 4 cost 2" to node 1 in the same round. Sending 16 instead of leaving it out is clearer, because the neighbor knows for sure the route isn't usable through us.
- Route poisoning: this happens after a failure. When the 3-4 link broke, node 3 withdrew its route and told everyone "4 is 16." Then node 2 started sending "dest 4 cost 16" to both node 1 and node 3. That's in our poison test log at 1:45.

- Why loops with 3+ nodes still happen: say A, B, and C are all connected to each other, and C is connected to D. A and B both reach D through C. If the C-D link breaks, C tells A and B "16." But B's route went through C, not A, so B's earlier advertisement to A said the real cost (2). If A hears B's old advertisement before B gets C's bad news, A switches to "D through B." Now A → B → C → A is a loop. Poison reverse only stops two neighbors from pointing at each other. This loop goes through 3 nodes, so it isn't caught. The costs keep going up until they hit 16, and the TTL makes sure no single packet loops forever.

4. Does your implementation necessarily produce symmetric forward and reverse paths? Explain how topology, equal-cost choices, and convergence affect the answer.

- No. Each node picks its own next hop by itself, so the path from A to B and the path from B to A don't have to match.
- Topology: if there are two paths with the same cost (like a square 1-2-4 and 1-3-4), node 1 might reach 4 through 2 while node 4 reaches 1 through 3.
- Equal-cost choices: on a tie we keep whichever route we learned first. Which one comes first depends on when each advertisement shows up, and that's random because of our timer jitter. So the two ends of a path can pick different sides of the square.
- Convergence: while routes are still updating, one direction can switch to the new path before the other one does. In our alternative path test, node 3 first reached node 2 through node 5 (cost 3), and only later switched to the path through node 1 (cost 2).
- We only use links that work both ways (neighbor discovery needs the reply to come back), so a path is never one-way just because the radio only works in one direction.
- In our line tests the paths were symmetric, but that's because a line only has one possible path.

5. What happens when a DV advertisement is lost or malformed? Explain how your periodic updates, validation, packetization, and expiration policy address these cases.

- Lost: nothing bad happens right away. A missing advertisement just means "no new news." Routes don't time out until 6 rounds without a refresh, and every round we send the whole table again, so the next round fixes it. In our loss test, ping replies were getting lost, but the routing tables stayed correct the whole time. When a neighbor did get dropped from too many lost replies, it was added back 0.3 seconds later once a reply got through.
- Malformed: the link layer drops any packet that isn't the right size, and DV checks the protocol field and that the entry count is 9 or less before reading any entries. Any cost of 15 or higher counts as unreachable, so a weird big number can't break the math. We also ignore advertisements from nodes that neighbor discovery hasn't confirmed.
- Packetization: every entry is handled on its own. If the second packet of a split table is lost, the routes from the